would be able to guarantee that all of these operations could be performed in logarithmic time in the worst-case scenario.

Table 7.1 Comparative analysis of performance vs implementation for cache with n elements

	Array (unsorted)	Array (sorted)	Linked List	Hash Table	Balanced Tree
Storing an entry	O(1)	O(n)	O(n)	O(1) #	O(log n)
Finding entry by name	O(n)	O(log n) $	O(n)	O(1) #	O(log n)
Removing oldest entry	O(n) §	O(n)	O(1)	O(n)	O(log n)

(#) Amortized

(§) It requires O(1) **time to locate the entry but, considering a static array, after removing the leftmost or rightmost element, the rest of the elements must be shifted one position.**

($) Using binary search.

So, from table 7. 1, we could infer that:

- The tree would look the best compromise on the general case;
- The hash table would be the best choice if we know the size of the cache is big enough (and the insertion of new elements infrequent enough) to rarely require removal of the oldest entry;